

Digital Computing Systems (CESE4130)

2026/27

Lab 1 Manual

Voulina Kandreli
Thomas Maliappis
Zacharia Rudge
Shadid Hassan
Ignacio García Ezquerro
Mottaqiallah Taouil
Georgi Gaydadjiev

Table of Contents

Table of Contents	2
1. Lab 1	3
1.1. Setup	3
1.2. Content	3
Part 1: Asynchronous Encoder & Decoder	3
Part 2: Debouncing of electrical signals	7
Part 3: Finite State Machine (FSM)	8
BONUS: Linear Feedback Shift Register Random Number Generator	10
Sign-off	11
2. Appendix	12
2.1. Setup	12
2.2. Simulation & Synthesis.....	14
Simulation	14
Running Project on FPGA	16
2.3. Vivado Project Setup	19

1. Lab 1

1.1. Setup

In this lab, you will be putting your Verilog skills to practice. You will write hardware descriptions for different applications and synthesize and implement these designs using Xilinx Vivado. Then, after synthesis and implementation, you will generate a bitstream of your design to upload to an FPGA board. On the FPGA board, you will test your designs.

During the lab, you will be using Vivado software. Vivado is available on the lab computers, you do not need to install it locally, but you can do it if you prefer it that way. Installation instructions are available in [Vivado Setup](#).

If you are new to Verilog or Hardware description languages, it is recommended that you go through the [Verilog Tutorial](#) before the lab. This will help you understand the workflow of how a hardware description program is written and the main building blocks of a Verilog code.

1.2. Content

Download the Vivado project templates used in this exercise here: [Vivado Lab1](#)

This lab will be organized as follows:

- [Part 1: Asynchronous Encoder & Decoder](#)
 - [Lab Template](#)
 - [Step 1: Binary Display](#)
 - [Step 2: Cardinal Direction Decoder](#)
 - [Part 2: Debouncing of electrical signals](#)
 - [Step 1: Binary Counter](#)
 - [Step 2: Adding Debouncing](#)
 - [Step 3: Converting to Gray Encoding](#)
 - [Part 3: Finite State Machine \(FSM\)](#)
 - [Step 1: Circular FSM](#)
 - [Step 2: High-Level FSM](#)
 - [Step 3: Introducing Safe State of execution](#)
 - [BONUS: Linear Feedback Shift Register Random Number Generator](#)
 - [What are LFSRs](#)
 - [Lab Procedure](#)
-

Part 1: Asynchronous Encoder & Decoder

In this lab, you will use a binary rotary switch and a gray code rotary switch to perform some basic asynchronous decoders.

Learning Goals of the Lab

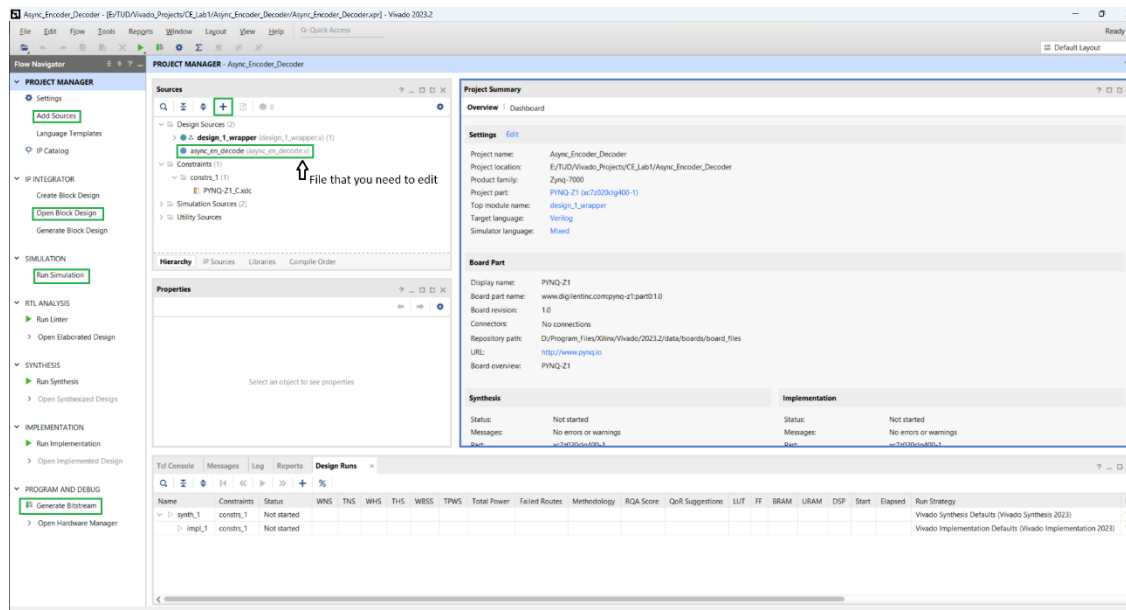
1. Introduction to Vivado and Vivado toolchain

2. Understanding block diagrams, inputs, and outputs of the FPGA
3. Implementing Asynchronous logic

Lab Template:

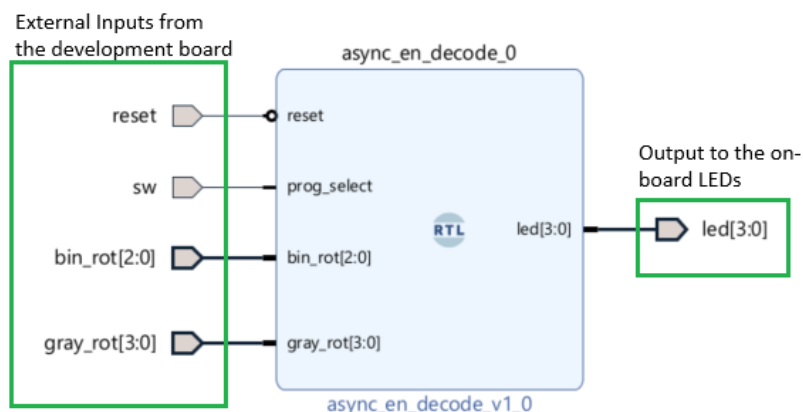
For all the exercises in this lab, you will receive a basic template for you to work on, except the *bonus* exercise. For this lab, we describe the components in the template. From the next exercises, you are expected to explore the template on your own.

To start the lab, launch the Vivado application. Open the project Async_Encoder_Decoder in Vivado. It will open the following screen.



Above you can see the file `async_en_decode.v`, which needs to be edited to complete the assignment. Other functions that you will be using in the lab are also highlighted.

You can now Open the Block Diagram. A block diagram is a visual representation of all the interconnected IPs and Verilog blocks with each other and with external components. The block diagram for this project is shown below.



There are 3 main parts to the above block diagram. 1. Inputs coming into the block diagram from the board. These include: - Reset switch: Mapped to SW1 - SW switch: Mapped to SW0 - Binary Rotary Encoder: Mapped to PMODB[0:2] - Gray Encoder Switch: Mapped to PMOD[4:7] 2. In the middle is the RTL code you will write in the Verilog file. 3. Finally, the output LEDs are mapped to the on-board LEDs

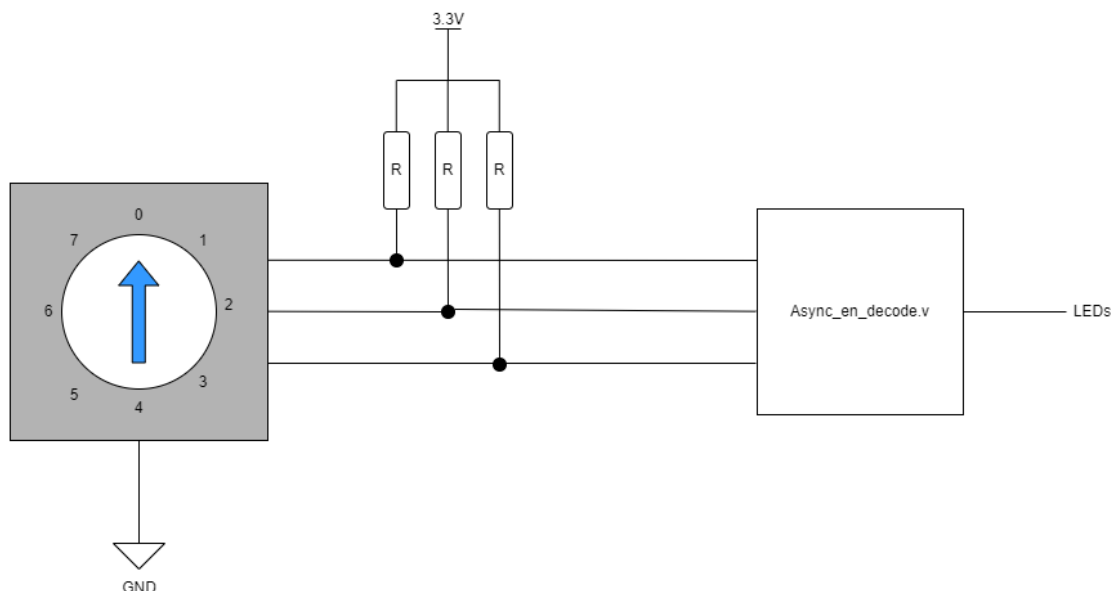
The above-mentioned component mapping is done in the constraints file. This file is located in Sources -> Constraints -> constrs_1 -> PYMQ-Z1_C.xdc. You can open and explore how the physical pins on the FPGA are mapped to various external components. These files are different for every development board, based on the on-board components.

Step 1: Binary Display

Now that you are a bit more familiar with the Vivado tool. Open the `async_en_decode.v` file. In this exercise, you will be mapping the binary input from the binary rotary switch to the on-board LEDs.

Follow the following steps to complete the assignment:

1. Make an asynchronous `always@(*)` block in the module
2. Implement the reset switch logic:
 - If the reset signal is on, your code should turn off all the LEDs
 - Only if reset is off, the rest of the code should be executed
3. Map the `bin_rot` input to the `led[2:0]` output. Refer to the circuit diagram below.
 - Note: The rotary switch is connected with pull-up logic

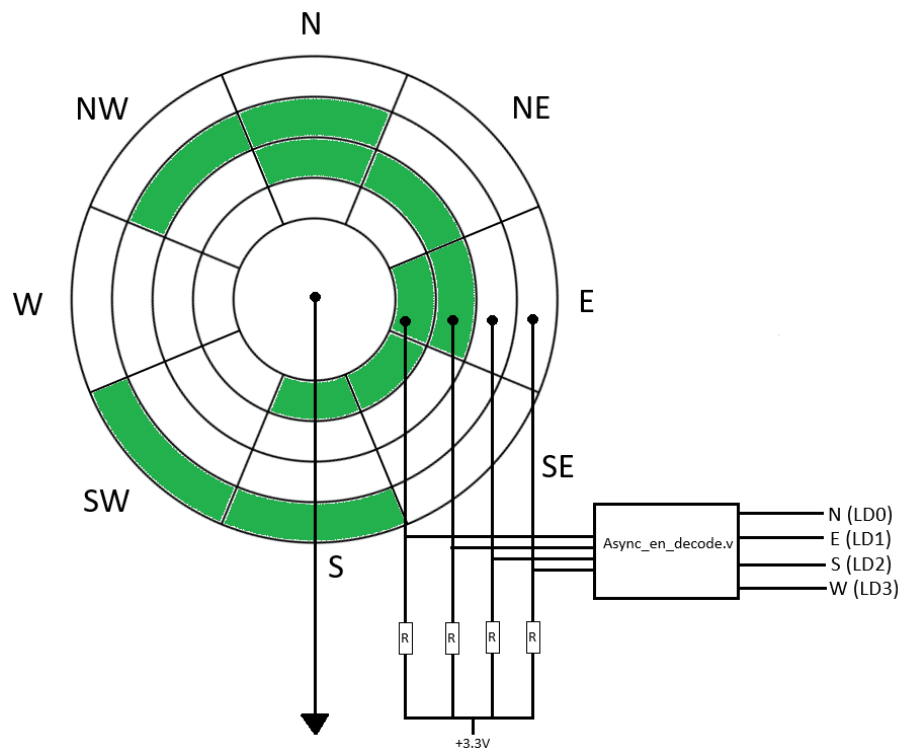


4. Make a testbench for your project and simulate.
 - You can find the steps to making your testbench in
 - Once you have the testbench working as expected you can move on to **Step 2**

The data sheet for the binary rotary switch being used: [Binary Rotary Switch \(S-8010\)](#)

Step 2: Cardinal Direction Decoder

In this exercise, you will interface with the gray rotary switch and convert the inputs from the gray rotary switch into Cardinal directions.



As you can see from the above image, you have to translate the rotary switch output into cardinal directions. For example, if the wheel is in the North position, only LD0 should be on and if the wheel is in the South-West direction, then both LD2 and LD3 should be on.

Follow the following steps to complete the assignment:

1. Since you are going to use the same project for 2 programs, you need to use the `prog_select` switch
 - When the switch is UP the binary display from *Step 1* should be executed
 - When the switch is DOWN the Cardinal Direction Decoder should be executed
2. Implement cardinal direction decoder logic in `async_en_decode.v`
3. Modify the testbench to test cardinal direction decoder logic
4. Once successfully tested, generate bitstream, and program the FPGA to test binary display

The data-sheet for the gray rotary switch being used: [Grey Rotary Switch](#)

Part 2: Debouncing of electrical signals

In this lab, you will work with the on-board buttons and make a binary and gray-encoded counter.

Learning Goals of the Lab

1. Interface with buttons on the board
2. Use Debouncing and understand its need
3. Implement synchronous logic
4. Programming and testing the FPGA

Step 1: Binary Counter

In this step, you should implement a synchronous binary counter, and display the value using the on-board LEDs. When BTN0 is pressed, the counter should increment, and BTN1 is pressed the counter should decrement.

Note: Do not forget to implement the reset logic, similar to **Part 1**

1. Open the Debouncer project in Vivado
2. Implement synchronous logic to increment and decrement the counter and display it on the LEDs
 - DO NOT modify the block diagram yet
3. Write a testbench for the module and simulate it using the simulation tool in Vivado
4. Generate bitstream and upload it to the FPGA

Did the counter work as expected? Read more about what is debouncing and why do we need it: [Debouncing](#)

Step 2: Adding Debouncing

Here, we introduce debouncing logic in our project. This should remove all issues with your program getting multiple button presses.

1. Open the Block-Diagram
2. Disconnect the *encoder* A & B inputs from the buttons
3. Connect the inputs to the btn_out of the debouncer block
4. Simulate to verify that you have made the connections correctly
5. Generate the bitstream and program the FPGA

Step 3: Converting to Gray Encoding

In this step, you will convert your binary value into a gray encoded value and display it on the LEDs

1. Add an additional asynchronous block to convert the binary value into a gray encoded value
 - Use an internal register to store your binary value
 - You can use a look-up table for the conversion or use the gray encoding equations for 4-bit numbers

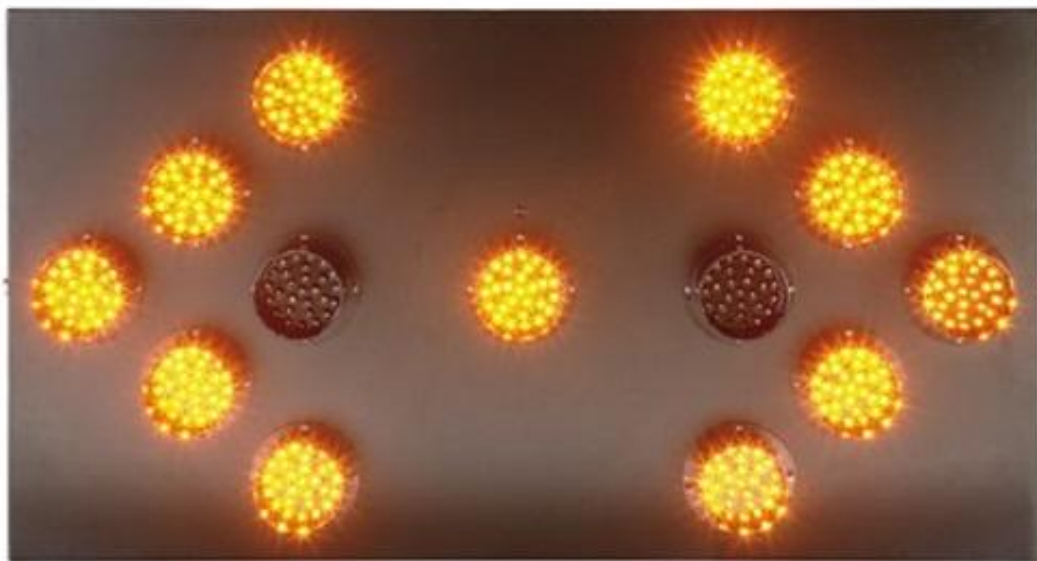
2. Simulate to verify that you have made the encoding correctly
3. Generate the bitstream and program the FPGA

Note: You can refer to [Gray Encoding](#).

Part 3: Finite State Machine (FSM)

In this part, you will be implementing an FSM for a Road Safety Sign. The particular sign that you need to implement has the following modes:

1. Arrow Point Left
2. Arrow Point Right
3. Blinking Lights Warning Mode
4. Safe Mode



Learning Goals of the Lab

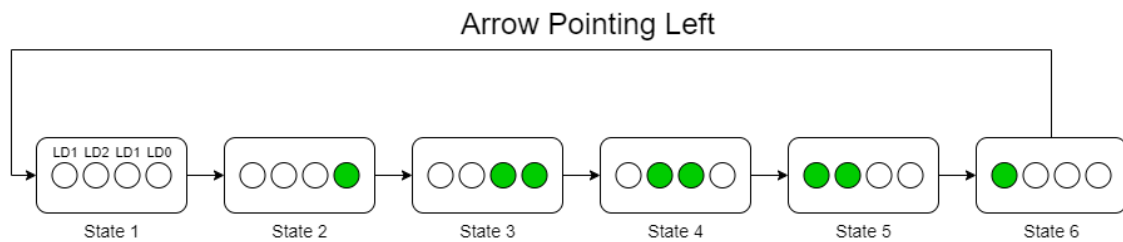
1. Understanding and Implementing a Finite state machine
2. Understanding different components and types of FSMs

To complete this exercise, use the `Road_Sign` provided in the template folder. Open it using Vivado. You will need to modify `Road_Sign\Road_Sign.srcs\sources_1\new\road_sign.v` to implement the encoder.

Step 1: Circular FSM

First, you will implement only the **Arrow Pointing Left Mode**. In this mode, the LEDs light up in a sequence moving towards the left. This is used to let the people know to divert to the left side of the road.

Since we have only 4 LEDs on the board, a pattern of running left LEDs is used to point left. For this, you will implement a 6-state circular FSM that has the following states:

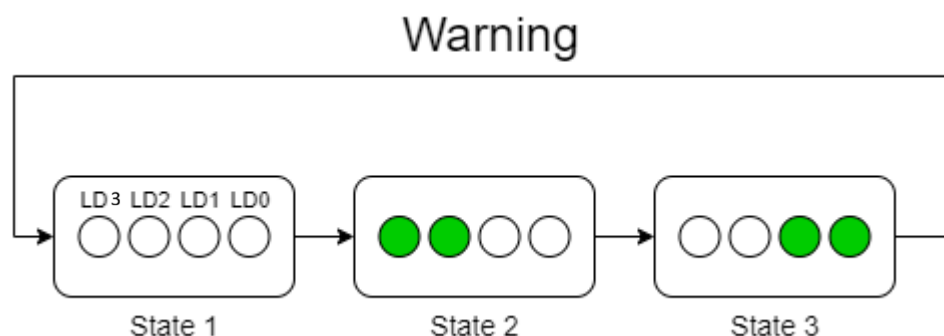
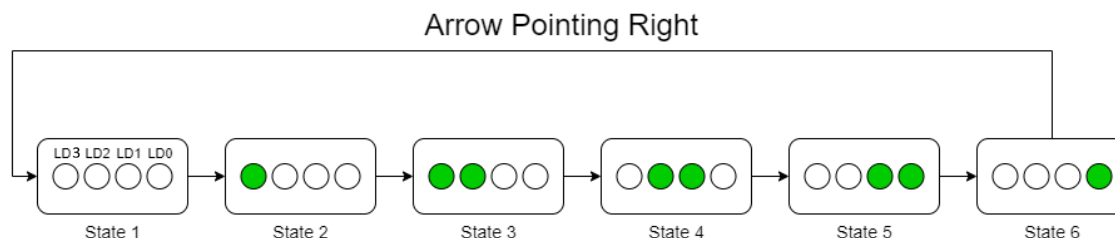


For the FSM to switch between the states, we have a time constraint. Every state transition should take place after 500ms. To achieve this, you can use the provided clock which is of 125MHz, and divide it appropriately so that the state update happens when expected.

Step 2: High-Level FSM

Once you have Point Left working correctly, you need to implement a higher-level FSM to control which mode the Road Sign is in.

To complete this step follow the following steps: 1. Make an independent 4-state FSM - The 4 states are Point-Left; Point-Right; Warning; Idle 2. The FSM state change is triggered by the 4 on-board buttons. When the corresponding button is pressed the FSM should enter that mode - Point-Left: BTN0 - Point-Right: BTN1 - Warning: BTN2 - Idle: BTN3 3. Before implementing the functionality of these states, verify that the high-level FSM is working as expected - Use the RGB_LED1 to verify your functionality (Map 1 color to each state) 4. Duplicate the circular FSM used before and use it for Arrow Point-Right and Warning. As shown in the below figures



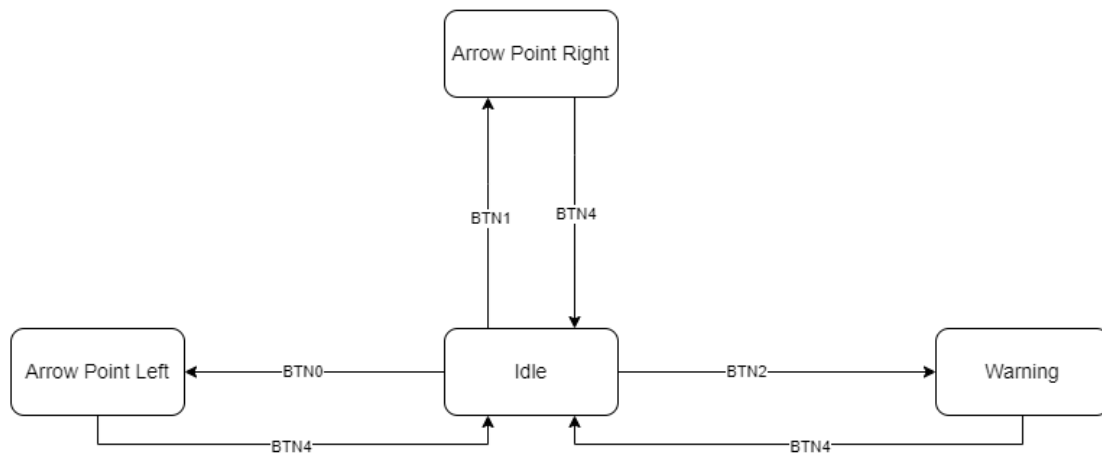
5. In the Idle state ensure that all the LEDs are turned OFF.

Step 3: Introducing Safe State of Execution

In the High-Level FSM implemented in the last step, any state can enter any other state. Which is a very unsafe way to transition between states. Since the road sign is deployed in a hazardous outdoor environment like a road-work site, it is important to

have an additional safety measure. This can prevent accidental changes in the state of the sign, which can be hazardous on an active highway for example.

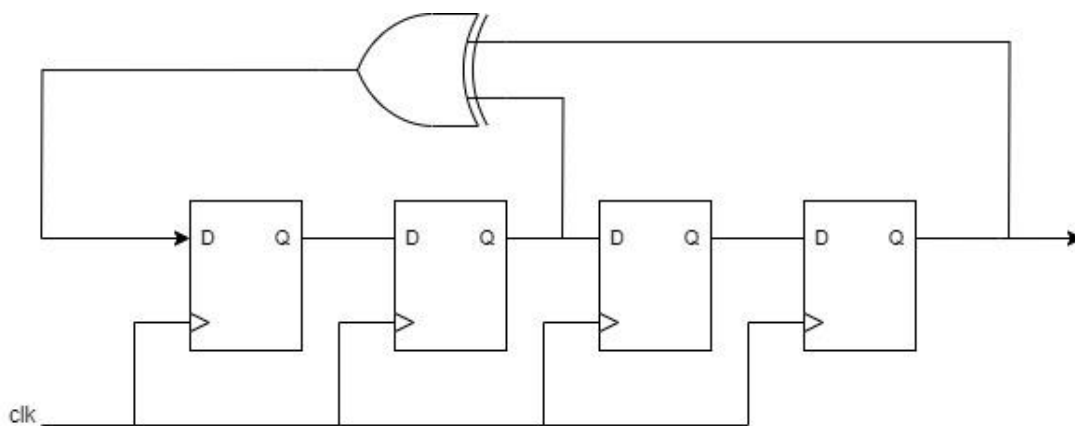
Thus, you need to add a constraint to the high-level FSM. You need to ensure, that if the FSM is in Point-Left, Point-Right, or Warning State it can only go to Idle state. And when in the Idle state it can enter any other state. A basic FSM diagram is presented below:



BONUS: Linear Feedback Shift Register Random Number Generator

What are LFSRs:

Linear Feedback Shift Register(LFSR) is a commonly used method to generate a set of pseudo-random numbers since it theoretically will be able to cover all the possible inputs for any n-bit circuits. A schematic of a typical LFSR is demonstrated in the figure below. A characteristic polynomial demonstrates the representation of the structure of the LFSR.



In this example, the characteristic polynomial is $P(x)=x^4+x^2+1$, which indicates that the first bit will be the XOR result of the 4th and 2nd bit. Repetition distance stands for the number of cycles that the two same generated pseudo-random numbers appear twice. To get the cycle counter, you will implement an internal counter to record the cycle. Seed it as the initial value of the internal register after you press the “reset” button.

Lab Procedure

Implement a 6-bit LFSR with the polynomial of $P(x)=x^6+x^5+x^3+1$, with three seeds, different seeds.

Seeds Stored

6'b00_0000

6'b01_1001

6'b10_1001

Follow the steps below to complete this lab: 1. Open Vivado and make a new project (no template is provided for this part) - Instructions on making a new Vivado project can be found in [Vivado Project Setup](#). 2. Implement a LFSR logic with the polynomial $P(x)=x^6+x^5+x^3+1$ in the project 3. Add logic to check when the LFSR loops back and starts repeating the sequence - We recommend having a repetition signal as an output, it can go high when the LFSR value is the same as the seed value 4. Write a testbench to test the LFSR Implementation 5. Simulate the LFSR using the 3 seeds mentioned above - Record the repetition distances for all three seeds

Sign-off

To keep the sign-off process efficient, we check all sections at once at the end. Please only call a TA once you have completed all mandatory parts (1, 2 and 3), with the FPGA connected and Vivado's Hardware Manager open. If you intend to do the bonus, finish that first as well, so everything can be signed off in one go. Make sure the work from earlier parts (generated bitstreams) is still available to demonstrate. Both group members must be present and able to answer questions during the sign-off.

2. Appendix

2.1. Setup

Please ensure that you have Lab Setup completed **before** the lab

Vivado Installation

To run the labs it is recommended to install Vivado and Vitis (you will also need them in the Processor Design Project in Q4), but if you absolutely cannot install them, contact the TAs so they can arrange access to a remote server. The entire installation requires between 50 to 60 GB of disk space.

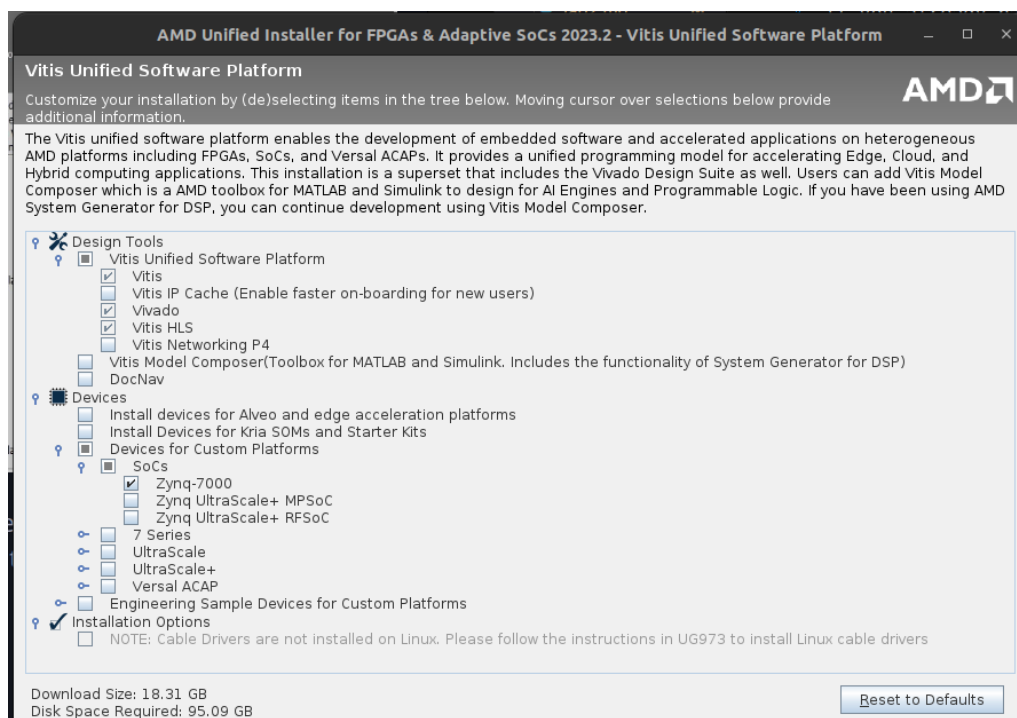
Installers are available for Linux and

Windows: <https://www.xilinx.com/support/download>. The coming lab was tested in version 2023.2 (and likely works in 2024.1). After you click on download, you need to create an account or log in if you already have one. On Linux do not forget to allow running `chmod a+x`

FPGAs_AdaptiveSoCs_Unified_2023.2_1013_2256_Lin64.bin and ./FPGAs_AdaptiveSoCs_Unified_2023.2_1013_2256_Lin64.bin

Once you run the installer:

1. Select Vitis
2. Select the minimal installation components as shown in the image below.
Windows: leave the last option (Cable drivers) checked.



3. Linux: install cable drivers by running:

```
cd ${vivado_install_dir}/data/xicom/cable_drivers/lin64/install_script/  
install_drivers/  
sudo ./install_drivers
```

Read more about Cable Drivers: <https://docs.amd.com/r/en-US/ug973-vivado-release-notes-install-license/Install-Cable-Drivers>.

After the installation is complete, ensure that you have Vivado 2023.2 and Vitis Classic 2023.2 installed.

If you face any issues with the installations please reach out to the TAs. You will not be able to start working on Lab 1 without the software completely installed. So you must install it before the lab so you don't lose time during the lab.

Installing a Serial Terminal

Only required for Lab 2, but recommended you install before the labs

A serial terminal is required to display the data sent back from the FPGA. There are many options for a serial terminal, any one of them is acceptable. A few recommended programs are listed below:

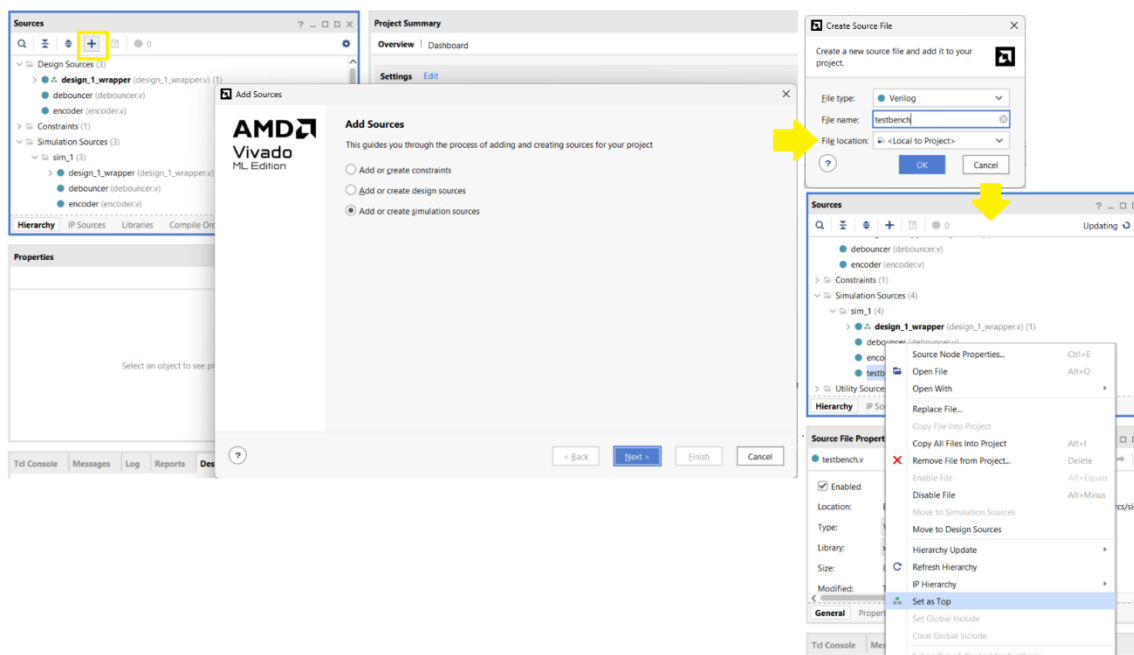
1. [Putty](#) (Suggested for new installs)
2. [MobaXTerm](#)
3. [Ardunio IDE](#)
4. [Termite](#)

2.2. Simulation & Synthesis

Simulation

Simulation is an important step when working on any FPGA project. It lets you test your code with custom inputs and access all the variables within your code. Additionally, for bigger projects, simulation has a much faster turn-around time rather than synthesizing your design and running it on the FPGA.

To simulate your project: 1. Create a new testbench file - With the project Open, select Add Sources or press Alt + A; select Add or create simulation sources - Select Create File, name the file testbench, then Finish - Expand Simulation Sources in the Sources window. - *Right-click* on your testbench file and select Set as Top. Your testbench file name should be written in **bold**



2. Write your test-bench
 - Create an Instance of the module you want to test.
 - Add code to modify the inputs to check various cases
 - If the module under test has synchronous logic, simulate a clock signal in your testbench

```
// General testbench example
`timescale 1ns / 1ps
```

```
module testbench;
    reg clk;      // Inputs as reg and outputs as wire
    reg reset;
    reg [3:0] input_signal;
    wire [3:0] output_signal;

    your_module uut ( // Instantiate the unit under test
        .clk(clk),
```

```

        .reset(reset),
        .input_signal(input_signal),
        .output_signal(output_signal)
    );

    initial begin // Clock generation block
        clk = 0;
        forever #5 clk = ~clk; // Invert clock signal every 5
time units
    end

    initial begin // Stimulus block
        reset = 1; // Initialize inputs
        input_signal = 0;

        #10; // Wait to un-toggle reset
        reset = 0;

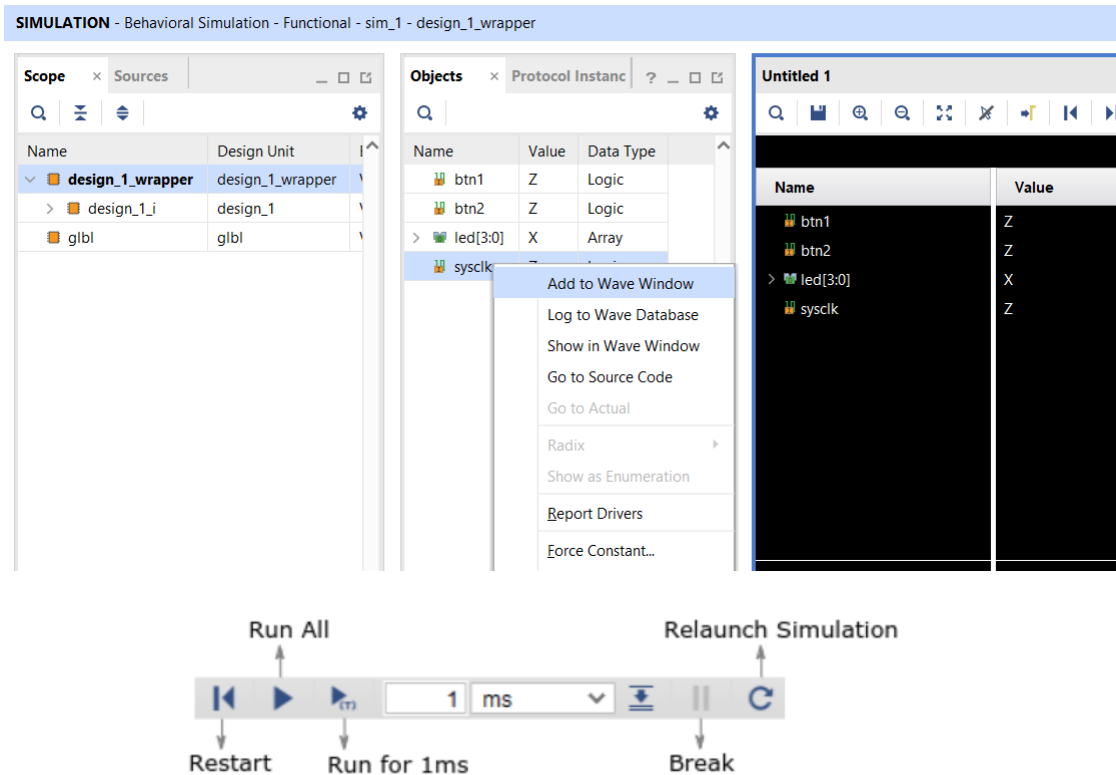
        #10 input_signal = 4'b0001; // Test inputs
        #10 input_signal = 4'b0010;
        #10 input_signal = 4'b0100;
        #10 input_signal = 4'b1000;

        #50; // Wait and finish
        $finish;
    end
endmodule

```

3. Simulating your design

- Launch the simulation by selecting, Flow Navigator -> SIMULATION -> Run Simulation -> Run Behavioral Simulation.
- When the simulation you defined in your testbench is complete, open the waveform viewer to view your signals.
- You can add more signals to the Wave Window. You can find more by expanding the Sources on the left side of the window. After you add a new signal you must reload the simulation to see the new signal changing.
- Use the controls on the top taskbar to start/stop the simulation
- Do not forget to resize your simulation window, so you can see the signals for the entire duration of the simulation



Running Project on FPGA

Once the project is working as expected in the simulation, you need to generate the bitstream to upload to the FPGA.

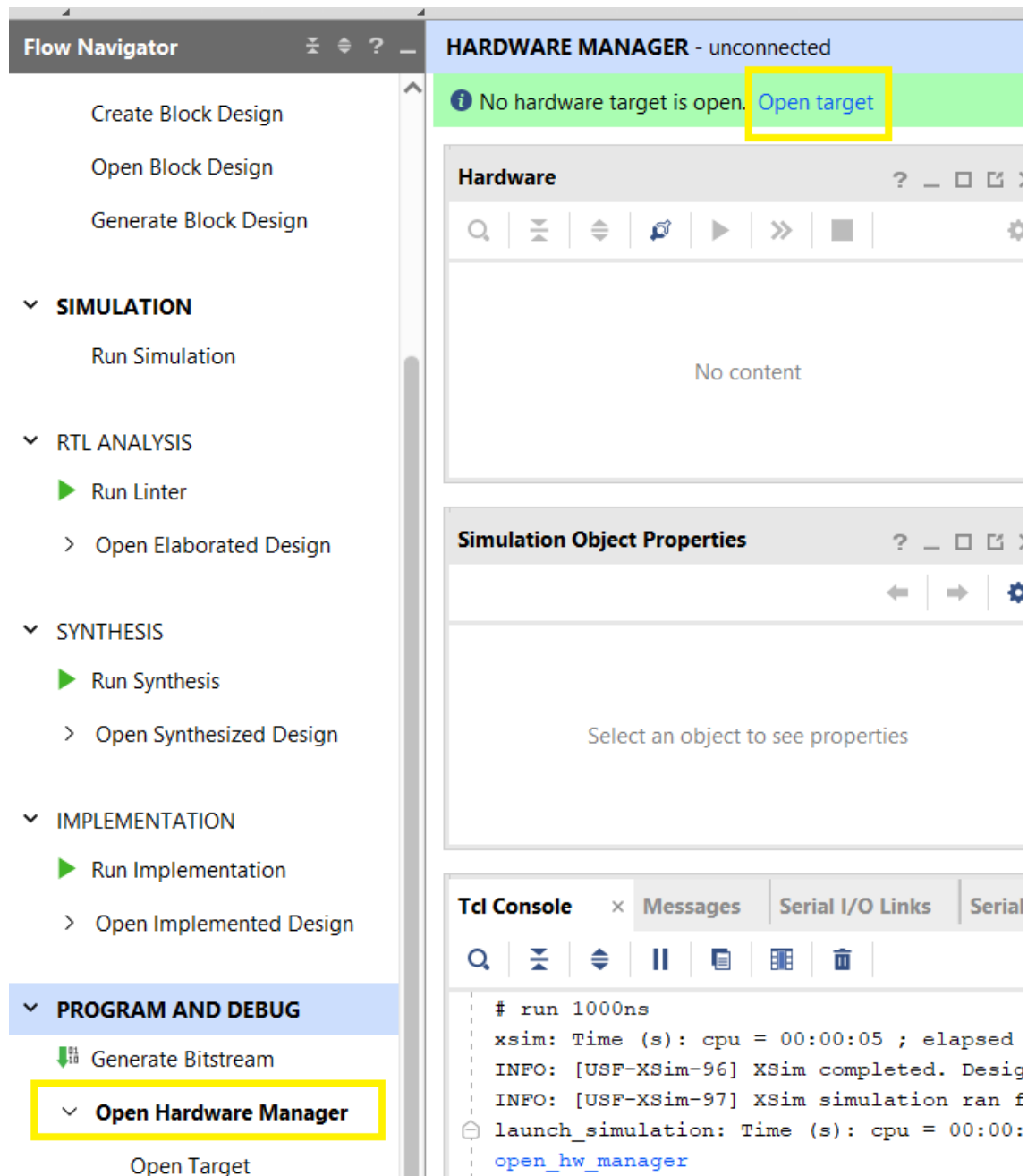
Run Program on the FPGA:

1. Generate Bitstream

1. Select Flow Navigator -> PROGRAM AND DEBUG -> Generate Bitstream
2. This will generate the bitstream for your project. This step may take up to 5 minutes to complete. So prefer the simulation to verify the correctness of your code.

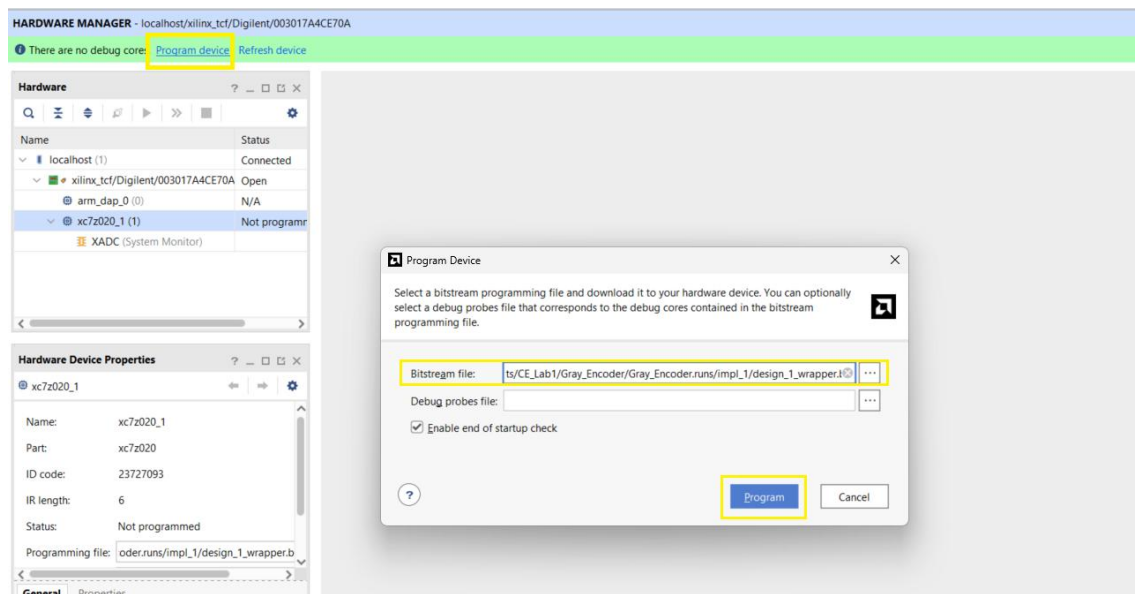
2. Connecting to the FPGA

1. Connect the FPGA to the Laptop using the USB Cable and turn ON the FPGA switch if it is OFF
2. In Vivado select Flow Navigator -> PROGRAM AND DEBUG -> Open Hardware Manager - Select Open target at the top of the window, then select Auto Connect



3. Program the FPGA

- *Right-click* on the FPGA, select Program Device
- Select the bitstream that was generated in the previous step. It can be found in <project_name>\<project_name>.runs\impl_1.
- Program the FPGA



Your FPGA is now programmed and running the Verilog code that you wrote!

2.3. Vivado Project Setup

Creating a Project

To generate a new project in Vivado follow the steps:

1. Download the Board files from [here](#) and paste them into `{installation_location}\Xilinx\Vivado\2023.2\data\boards\board_files\`
2. Start Vivado, Select Create Project
3. Name your project and select Next
4. Select RTL Project and ensure, Do not specify sources at this time is selected
5. Select Boards and search for PYNQ-Z1; Select the board from the list and click Next
6. Click Finish and your project is **completed**

Setting up a Project

First you need to add the constraints file for the selected board. For the PYNQ-Z1 board follow the following steps:

7. Add the constraints file for the board from the template repo, PYNQ-Z1_Constraints.xdc. Press the + button in the Sources Tab or Press ALT+A
8. Select Add or create constraints, add the file downloaded in the previous step, and select Finish
9. The constraint file has all the available GPIOs, LEDs, Buttons, and Switches on the board. You need to uncomment the ones you will need for the project.
10. PYNQ-Z1 also provides a 125MHz clock, enable it and use it in your design.

Second, you will have to either add or create a new source file. This can be done similarly to the constraint file but by selecting, Add or create sources. A source file is a Verilog file, in which you write your code. Ensure that the inputs and outputs of the Verilog module are set correctly.

Third, you will have to create a new block diagram. Block Diagram is used to stitch different IPs and components together in your project. To create a block diagram select Flow Navigator -> IP Integrator -> Create Block Diagram. You can now add your Verilog source to this block diagram by right-clicking on the file and selecting Add Module to Block Diagram. If you want to connect any pins from your block to external GPIOs, you can right-click on them and select Make External or press Cntr + T.

Ensure that the names of the resources in the constraint file and the block diagram are the same.

Fourth, and finally, once your block diagram is completed *Right-click* on it and select Create HDL Wrapper. . . . This will make an additional Verilog file which will act as the top layer of your project connecting all the IPs and sources in the block diagram together.

Now the project can be simulated or bitstream can be generated.